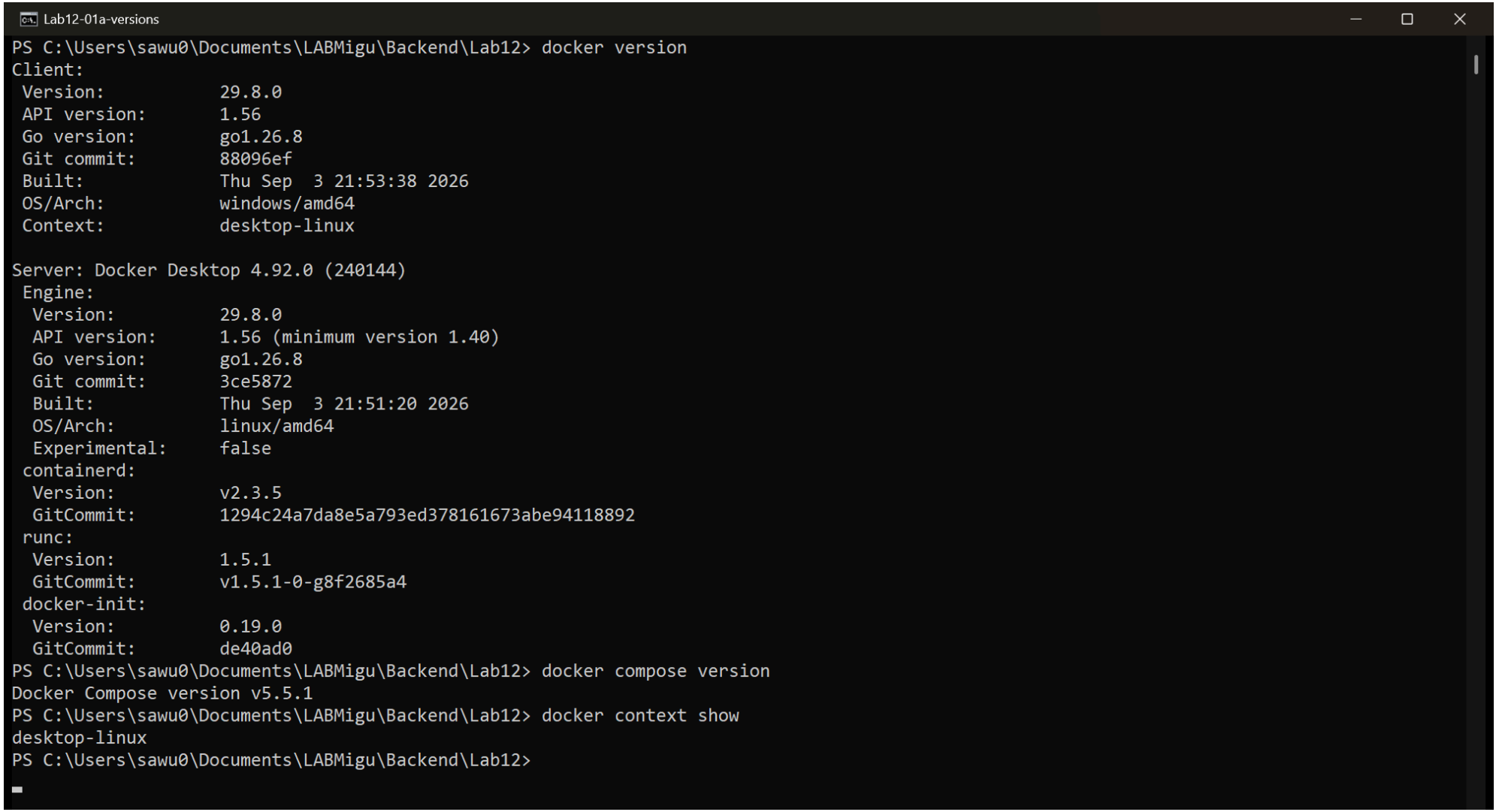
ITCS258 Lab 12: Compose and persistent data

Name: Pasit Sugrivaka
Student ID: 6788117
OS: Windows 11 Home (PowerShell 7)
Docker context: `desktop-linux` (Docker Desktop 4.92.0, Engine 29.8.0, Compose v5.5.1)
Own project: Hotel Booking System API (NestJS, Prisma, MySQL, Redis). Repository: https://github.com/Sor117/itcs258-lab12
My marker word is **Capri**. The reference row is `Lab12-6788117-Capri-wakes-up` and my project row is `Lab12-6788117-Capri-engram-suite`.

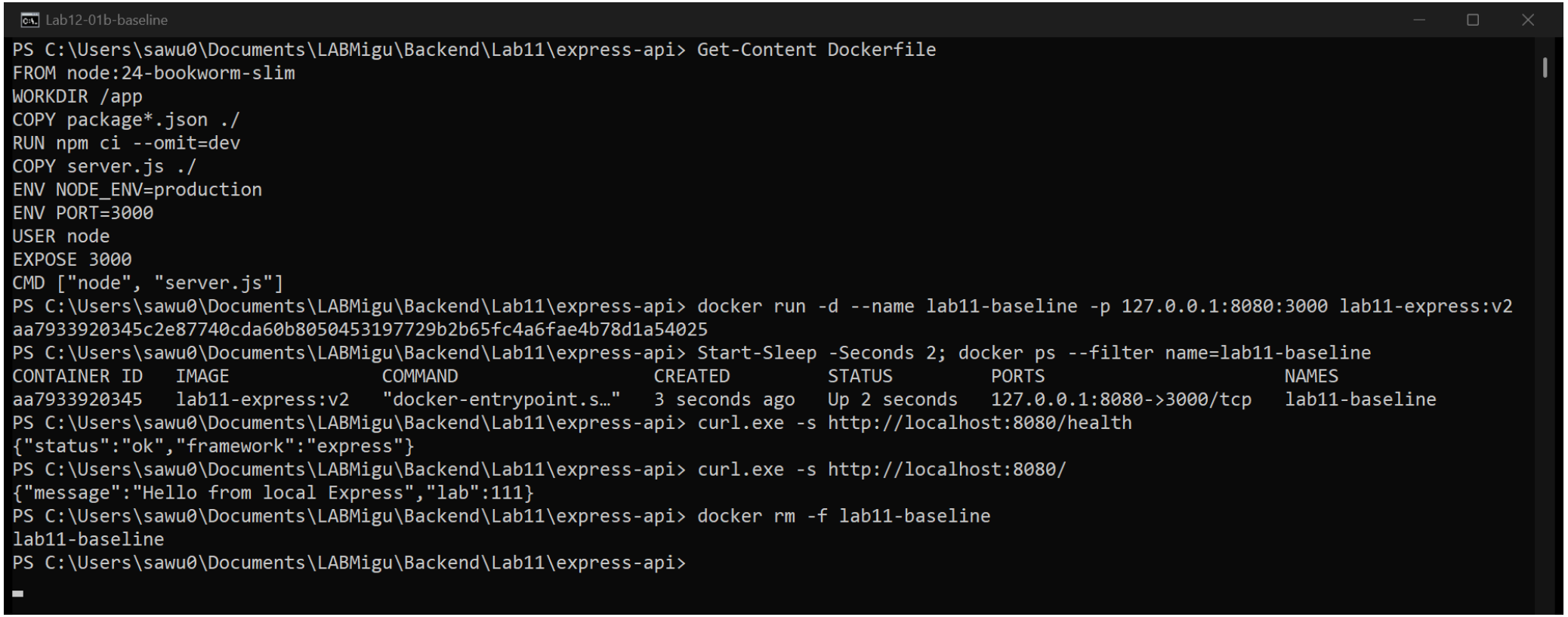
01 Lab 11 baseline and Compose readiness

What moves from `docker run` into Compose:

`docker build -t ...` becomes `build: .` (Compose builds the image and names it after the project).
`-p 127.0.0.1:8080:3000` becomes `ports: ["127.0.0.1:8082:3000"]` on the app service only.
`-e` / `--env-file` becomes the `environment:` block, with values read from `.env`.
`--name lab11-...` becomes the project name `lab12-express-6788117` plus the service names app and db.
`host.docker.internal` (Lab 11) is replaced by the service name db on the Compose network.
New in Compose: a named volume for the DB, healthchecks, and depends_on so the app waits for the DB.
`docker stop` + `docker rm` becomes `docker compose down`.



`docker version` shows both Client and Server (Engine 29.8.0), so the engine is running. Compose is v5.5.1 and the context is `desktop-linux`.



Baseline before adding a database: my Lab 11 Express image `lab11-express:v2`. The Dockerfile starts the app with `node server.js`, the container port is 3000 and the health route is `/health`. I ran it with `-p 127.0.0.1:8080:3000` and both requests worked, then I removed the container.

02 App + DB stack

Folder `Lab12/reference-express`. I added the pg package to the Lab 11 Express starter, plus `POST /notes`, `GET /notes`, and a `/health` route that runs a real SQL query. The app creates the `notes` table when it starts.

Lab12-02a-compose-file

```
PS C:\Users\sawu0\Documents\LABMigu\Backend\Lab12\reference-express> Get-Content docker-compose.yml
name: lab12-express-${STUDENT_ID:?set STUDENT_ID in .env}

services:
  app:
    build: .
    ports:
      - "127.0.0.1:8082:3000"
    environment:
      PORT: 3000
      APP_MESSAGE: ${APP_MESSAGE:-Hello from Compose Express}
      DB_HOST: db
      DB_PORT: 5432
      DB_USER: ${POSTGRES_USER}
      DB_PASSWORD: ${POSTGRES_PASSWORD}
      DB_NAME: ${POSTGRES_DB}
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "node", "-e", "fetch('http://127.0.0.1:3000/health').then(r => process.exit(r.ok ? 0 : 1)).catch(() => process.exit(1))"
]
      interval: 5s
      timeout: 3s
      retries: 10
      start_period: 5s
    restart: unless-stopped

  db:
    image: postgres:18
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      # postgres 18+ keeps data under /var/lib/postgresql/18/docker, so mount the parent
      - capricorn_vault:/var/lib/postgresql
    healthcheck:
      # -h 127.0.0.1 so the temporary socket-only server during first init doesn't count as ready
      test: ["CMD-SHELL", "pg_isready -h 127.0.0.1 -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: unless-stopped

volumes:
```

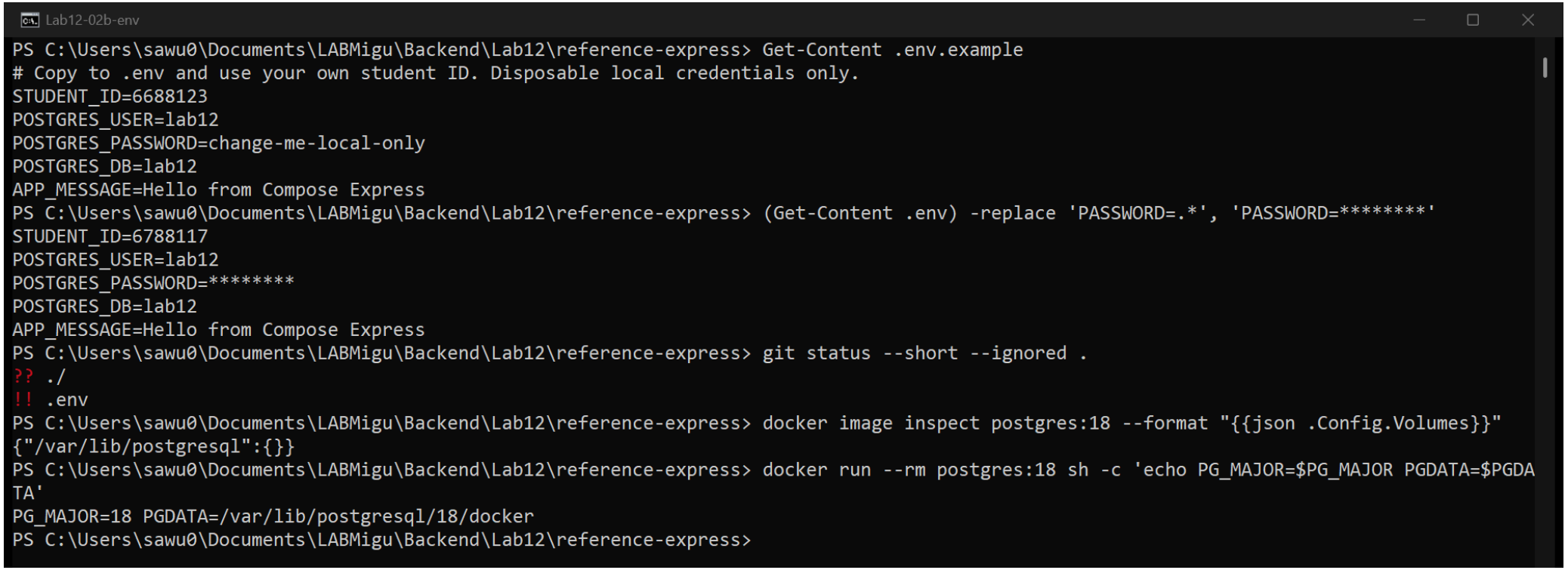
Lab12-02a-compose-file

```
services:
  app:
    build: .
    ports:
      - "127.0.0.1:8082:3000"
    environment:
      PORT: 3000
      APP_MESSAGE: ${APP_MESSAGE:-Hello from Compose Express}
      DB_HOST: db
      DB_PORT: 5432
      DB_USER: ${POSTGRES_USER}
      DB_PASSWORD: ${POSTGRES_PASSWORD}
      DB_NAME: ${POSTGRES_DB}
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "node", "-e", "fetch('http://127.0.0.1:3000/health').then(r => process.exit(r.ok ? 0 : 1)).catch(() => process.exit(1))"
]
      interval: 5s
      timeout: 3s
      retries: 10
      start_period: 5s
    restart: unless-stopped

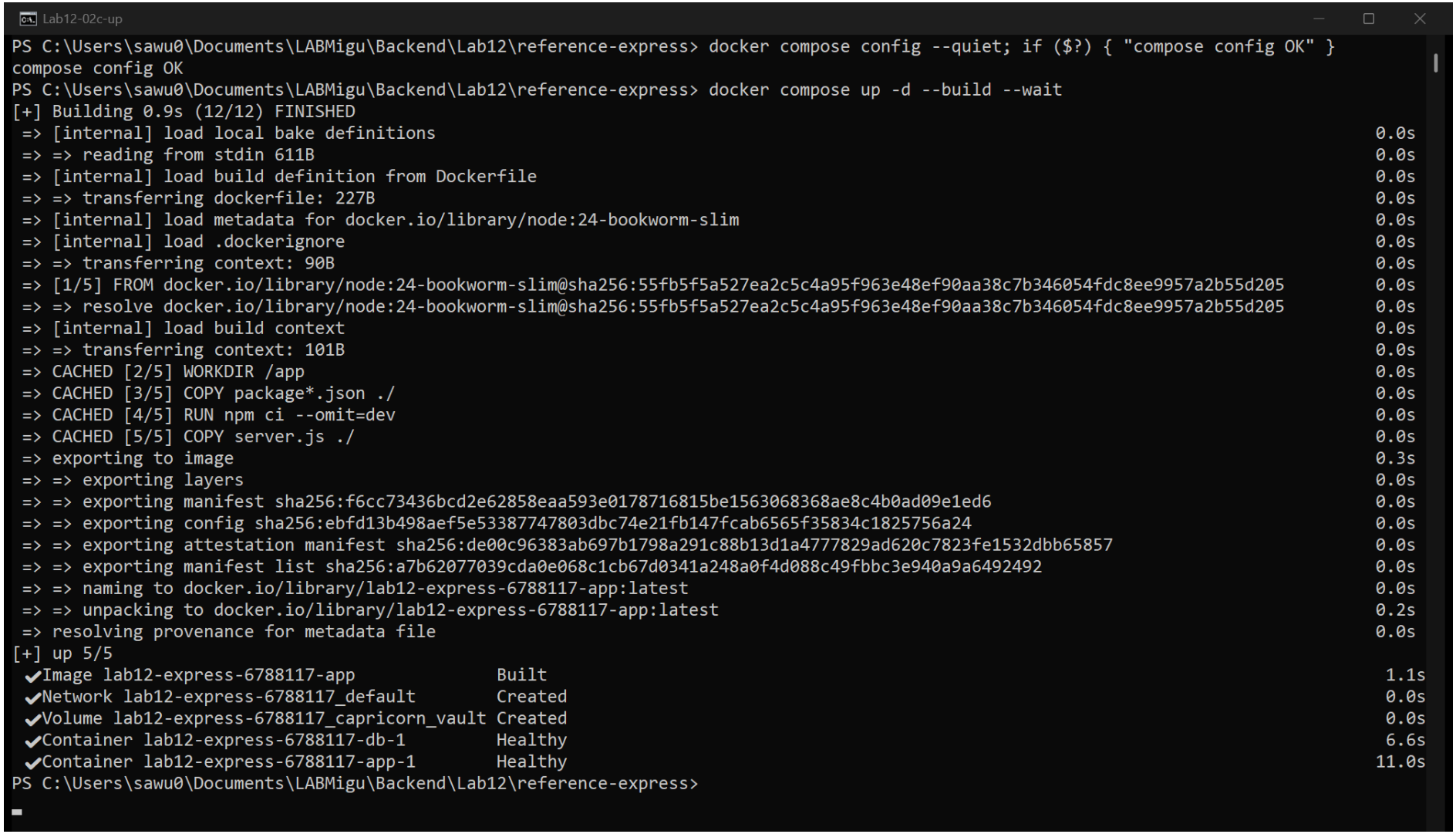
  db:
    image: postgres:18
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      # postgres 18+ keeps data under /var/lib/postgresql/18/docker, so mount the parent
      - capricorn_vault:/var/lib/postgresql
    healthcheck:
      # -h 127.0.0.1 so the temporary socket-only server during first init doesn't count as ready
      test: ["CMD-SHELL", "pg_isready -h 127.0.0.1 -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: unless-stopped

volumes:
  capricorn_vault:
PS C:\Users\sawu0\Documents\LABMigu\Backend\Lab12\reference-express>
```

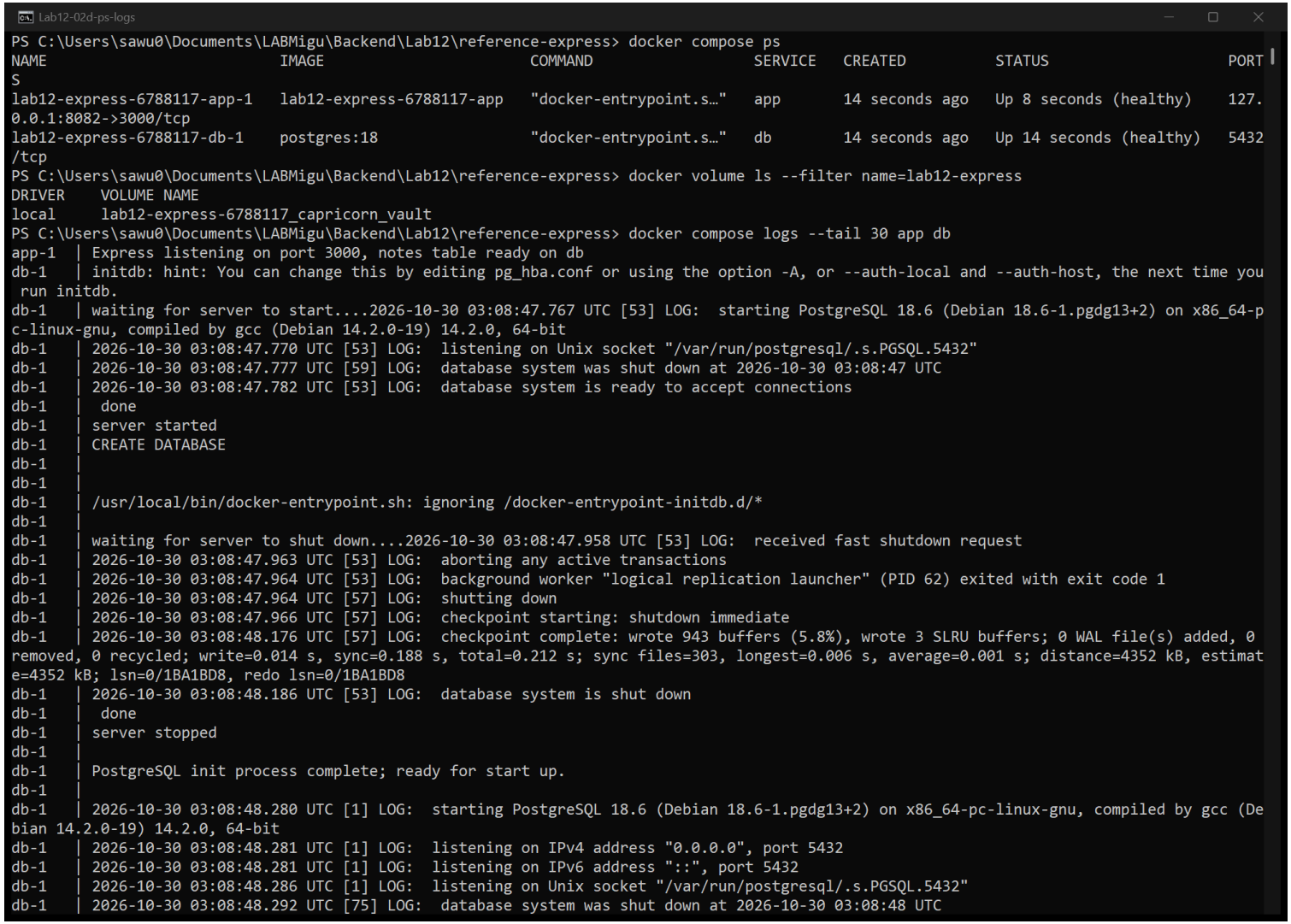
`docker-compose.yml`: exactly two services, app and db. The project name comes from `STUDENT_ID`. Only app publishes a port. db has the named volume `capricorn_vault`, and app waits until db is healthy.

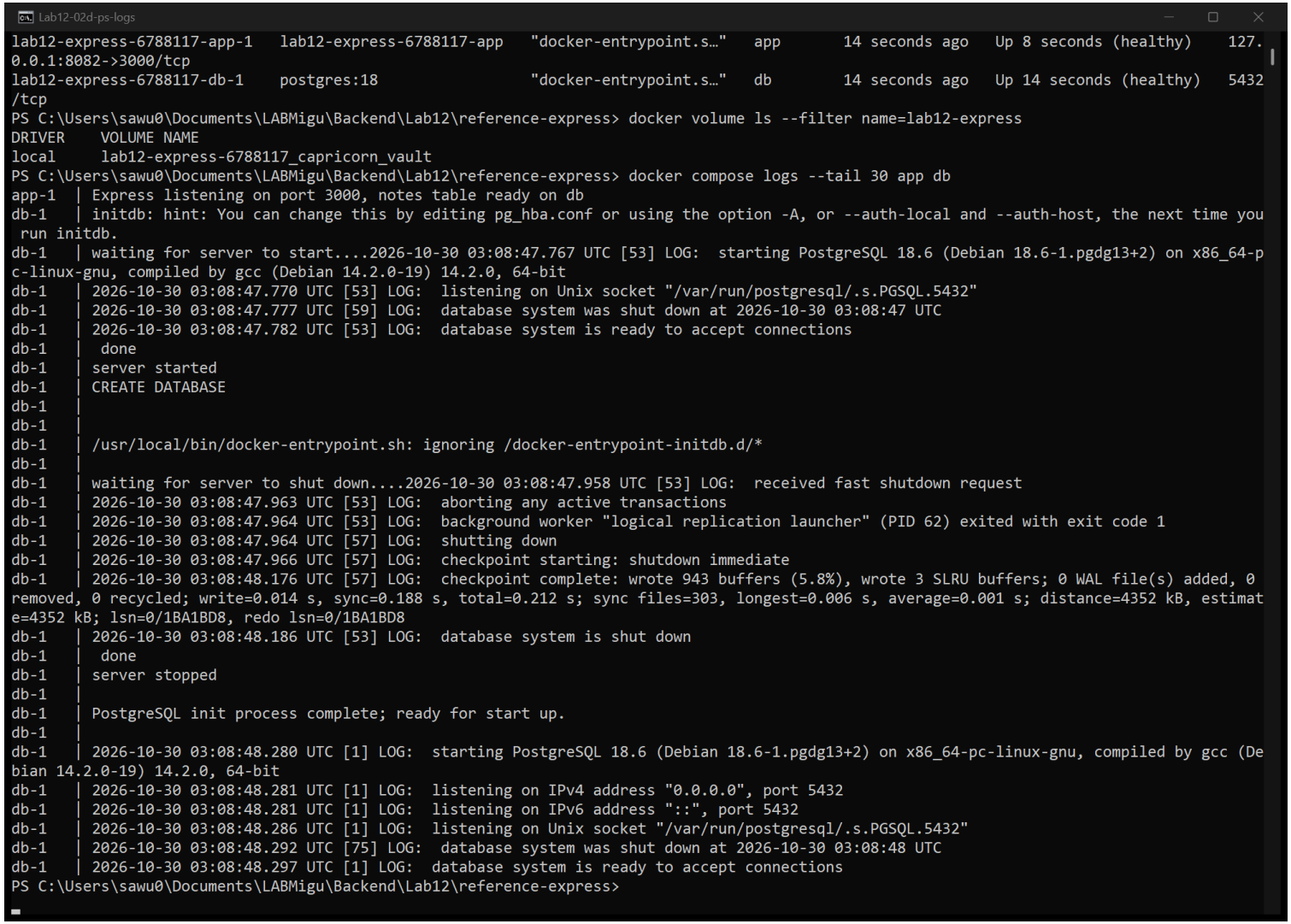


`.env.example` (tracked) and my real `.env` with the password hidden. `git status --ignored` shows `.env` as ignored (`!!`). The `postgres:18` image declares its volume at `/var/lib/postgresql` and keeps data in `/var/lib/postgresql/18/docker`, so I mount the volume on `/var/lib/postgresql`. Older guides use `/var/lib/postgresql/data`, which is wrong for major 18.

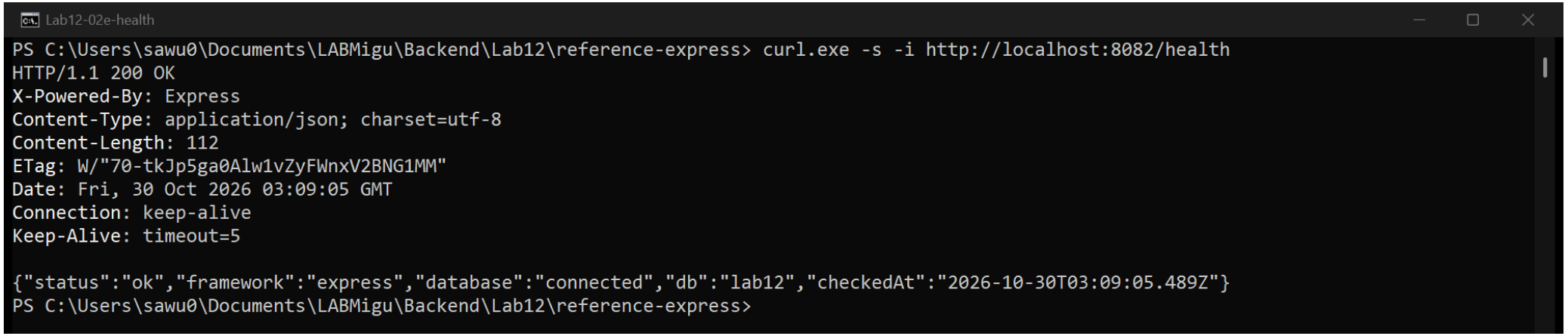


`docker compose config --quiet` passes, then `up -d --build --wait` builds the app and creates the network, the volume `lab12-express-6788117_capricorn_vault` and both containers. It returns only after both are Healthy.


Lab12-02d-ps-logs
PS C:\Users\sawu0\Documents\LABMigu\Backend\Lab12\reference-express> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORT
S
lab12-express-6788117-app-1 lab12-express-6788117-app "docker-entrypoint.s…" app 14 seconds ago Up 8 seconds (healthy) 127.
0.0.1:8082->3000/tcp
lab12-express-6788117-db-1 postgres:18 "docker-entrypoint.s…" db 14 seconds ago Up 14 seconds (healthy) 5432
/tcp
PS C:\Users\sawu0\Documents\LABMigu\Backend\Lab12\reference-express> docker volume ls --filter name=lab12-express
DRIVER VOLUME NAME
local lab12-express-6788117_capricorn_vault
PS C:\Users\sawu0\Documents\LABMigu\Backend\Lab12\reference-express> docker compose logs --tail 30 app db
app-1 | Express listening on port 3000, notes table ready on db
db-1 | initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you
run initdb.
db-1 | waiting for server to start....2026-10-30 03:08:47.767 UTC [53] LOG: starting PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-p
c-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
db-1 | 2026-10-30 03:08:47.770 UTC [53] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2026-10-30 03:08:47.777 UTC [59] LOG: database system was shut down at 2026-10-30 03:08:47 UTC
db-1 | 2026-10-30 03:08:47.782 UTC [53] LOG: database system is ready to accept connections
db-1 | done
db-1 | server started
db-1 | CREATE DATABASE
db-1 |
db-1 |
db-1 | /usr/local/bin/docker-entrypoint.sh: ignoring /docker-entrypoint-initdb.d/*
db-1 |
db-1 | waiting for server to shut down....2026-10-30 03:08:47.958 UTC [53] LOG: received fast shutdown request
db-1 | 2026-10-30 03:08:47.963 UTC [53] LOG: aborting any active transactions
db-1 | 2026-10-30 03:08:47.964 UTC [53] LOG: background worker "logical replication launcher" (PID 62) exited with exit code 1
db-1 | 2026-10-30 03:08:47.964 UTC [57] LOG: shutting down
db-1 | 2026-10-30 03:08:47.966 UTC [57] LOG: checkpoint starting: shutdown immediate
db-1 | 2026-10-30 03:08:48.176 UTC [57] LOG: checkpoint complete: wrote 943 buffers (5.8%), wrote 3 SLRU buffers; 0 WAL file(s) added, 0
removed, 0 recycled; write=0.014 s, sync=0.188 s, total=0.212 s; sync files=303, longest=0.006 s, average=0.001 s; distance=4352 kB, estimat
e=4352 kB; lsn=0/1BA1BD8, redo lsn=0/1BA1BD8
db-1 | 2026-10-30 03:08:48.186 UTC [53] LOG: database system is shut down
db-1 | done
db-1 | server stopped
db-1 |
db-1 | PostgreSQL init process complete; ready for start up.
db-1 |
db-1 | 2026-10-30 03:08:48.280 UTC [1] LOG: starting PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (De
bian 14.2.0-19) 14.2.0, 64-bit
db-1 | 2026-10-30 03:08:48.281 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
db-1 | 2026-10-30 03:08:48.281 UTC [1] LOG: listening on IPv6 address "::", port 5432
db-1 | 2026-10-30 03:08:48.286 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2026-10-30 03:08:48.292 UTC [75] LOG: database system was shut down at 2026-10-30 03:08:48 UTC



`docker compose ps` shows both containers healthy, and only the app has a host port (`127.0.0.1:8082->3000`). The DB logs show the first-run init (create the database, stop the temporary server) and then the real server listening on 5432. The app log says the notes table is ready on db.



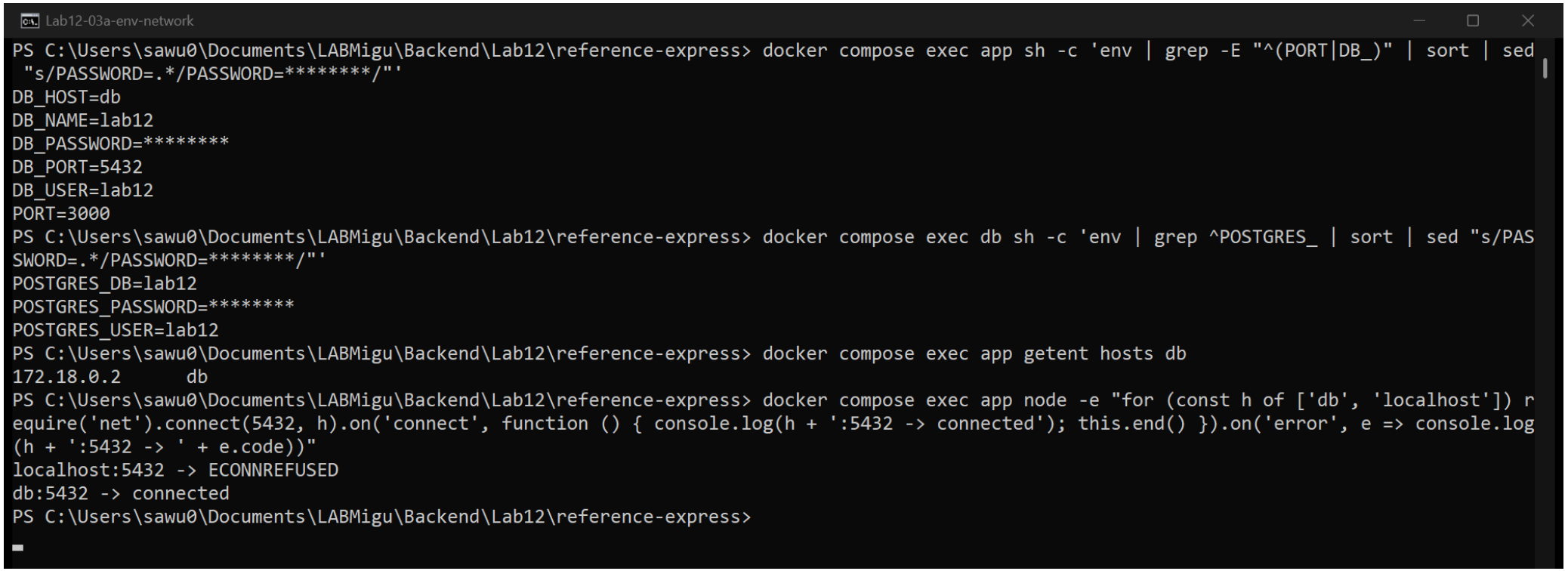
`GET /health` returns 200 with `"database":"connected"`. This comes from a query, not a fixed response.

Healthchecks. db uses `pg_isready -h 127.0.0.1`. On the first start, Postgres runs a temporary server that only listens on a Unix socket, so a socket check could say "ready" too early. Checking over TCP avoids that. app is healthy only when `/health` returns 200, which means it can query the DB.

Ports. Only app publishes a port, on `127.0.0.1:8082`. The DB does not need one because the app reaches it inside the Compose network, and keeping 5432 closed means nothing on my laptop or LAN can log in to it directly. I used 8082 so it does not clash with Lab 11's 8080/8081.

03 Configuration, networking and data

Two kinds of variables. Compose reads `.env` only to fill in `${...}` inside `docker-compose.yml` (interpolation). A variable in `.env` does not go into a container by itself. Each container gets only what its `environment:` block lists: app gets PORT, DB_HOST=db, DB_PORT, DB_USER, DB_PASSWORD and DB_NAME, and db gets POSTGRES_USER, POSTGRES_PASSWORD and POSTGRES_DB. STUDENT_ID is only used for the project name.



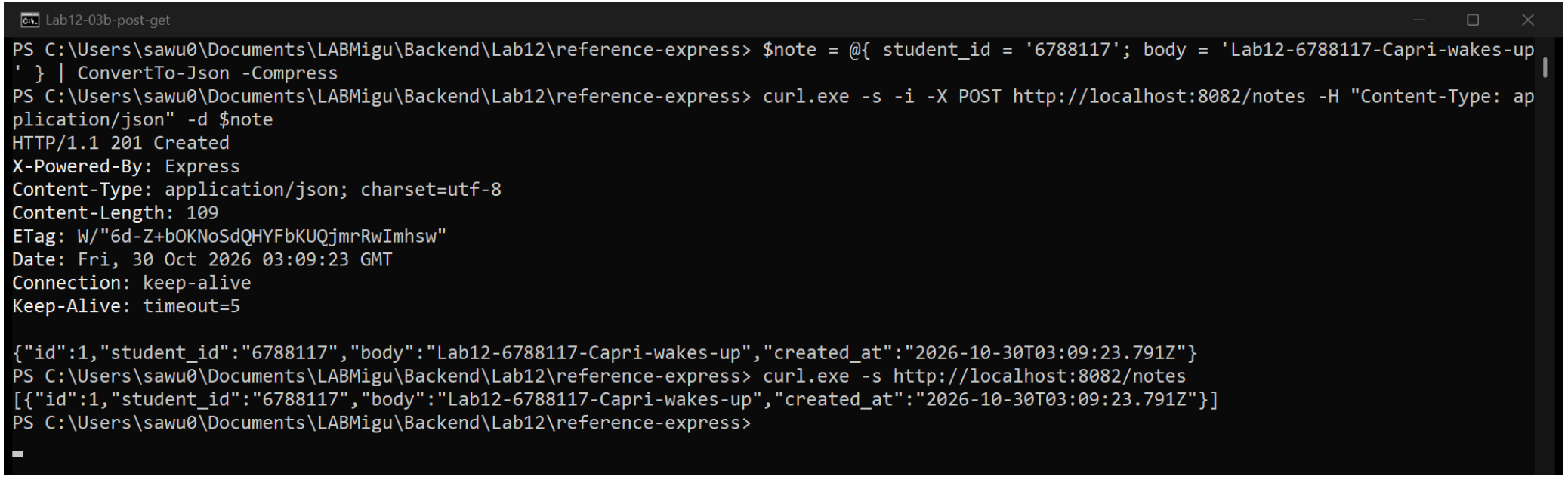
The variables inside each container (passwords hidden). Inside the app, db resolves to 172.18.0.2. A TCP test from the app container: `db:5432` connects, but `localhost:5432` is refused.

Connection path:

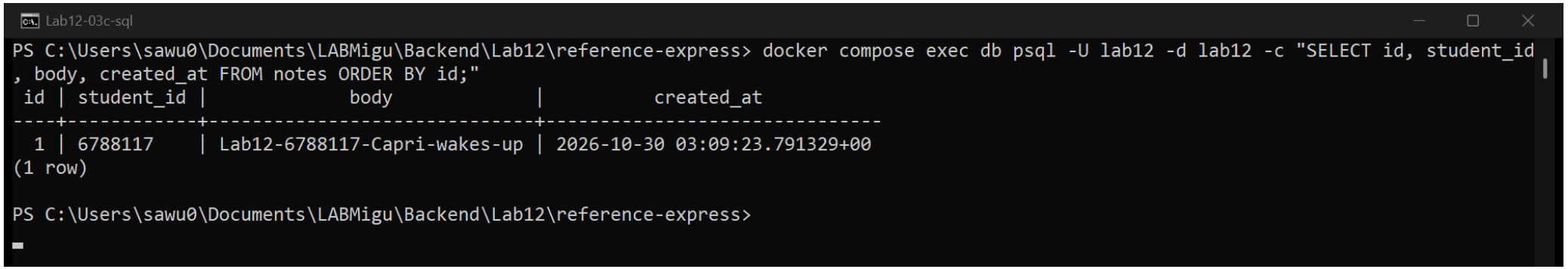
Browser / curl on my laptop → `localhost:8082` → (Docker port publish) → app container port 3000
app container → `db:5432` on the `lab12-express-6788117_default` network → db container (PostgreSQL)

My laptop uses `localhost:8082` because Docker publishes the app there. Inside the app container, `localhost` means the app container itself, and nothing listens on 5432 there, so `localhost` would be the wrong DB host. Compose gives every service a DNS name on its network, so the app uses db.

Testing tool: `curl.exe` and PowerShell (`ConvertTo-Json` for the body).



POST /notes with my student ID and marker returns **201 Created**. GET /notes returns the same row.

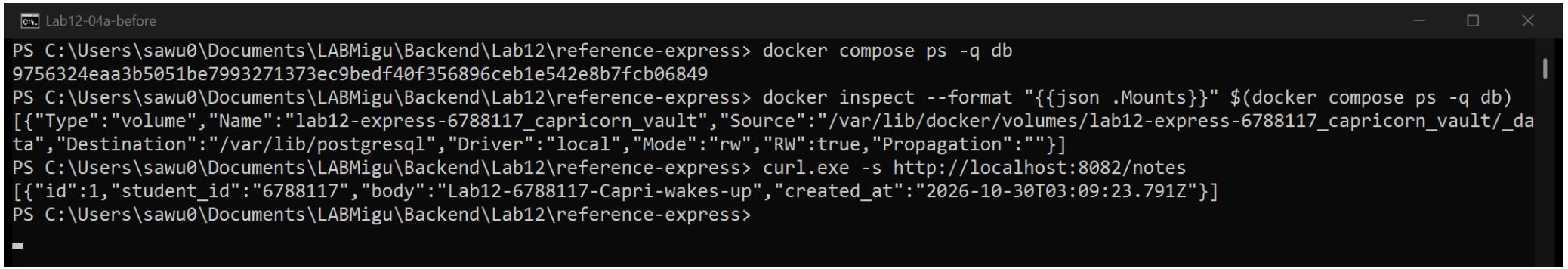


Independent check with psql inside the DB container. The row matches the API.

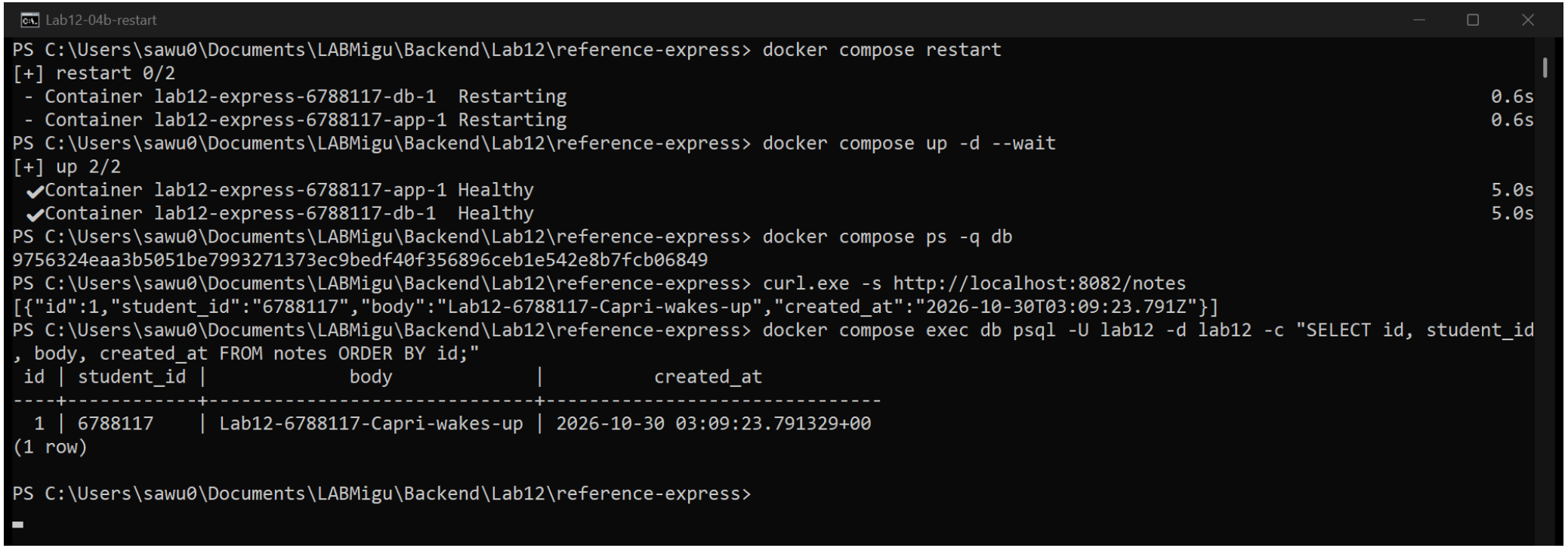
Original row: id 1, student_id 6788117, body Lab12-6788117-Capri-wakes-up, created_at 2026-10-30T03:09:23.791Z (API) = 2026-10-30 03:09:23.791329+00 (SQL). Same instant; the API shows milliseconds and SQL shows microseconds.

04 Persistence after replacement

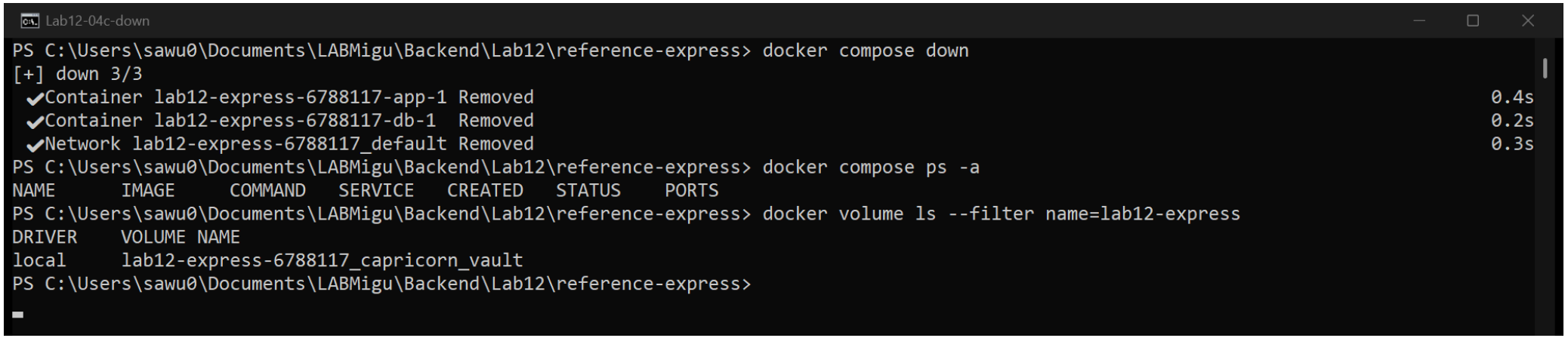
Test	DB container ID	Named volume	Record ID	Marker + created_at
Initial	9756324eaa3b	lab12-express-6788117_capricorn_vault	1	Lab12-6788117-Capri-wakes-up, 2026-10-30T03:09:23.791Z
After restart	9756324eaa3b (same)	lab12-express-6788117_capricorn_vault	1	Lab12-6788117-Capri-wakes-up, 2026-10-30T03:09:23.791Z
After down/up	9e3d5d0a6487 (new)	lab12-express-6788117_capricorn_vault	1	Lab12-6788117-Capri-wakes-up, 2026-10-30T03:09:23.791Z



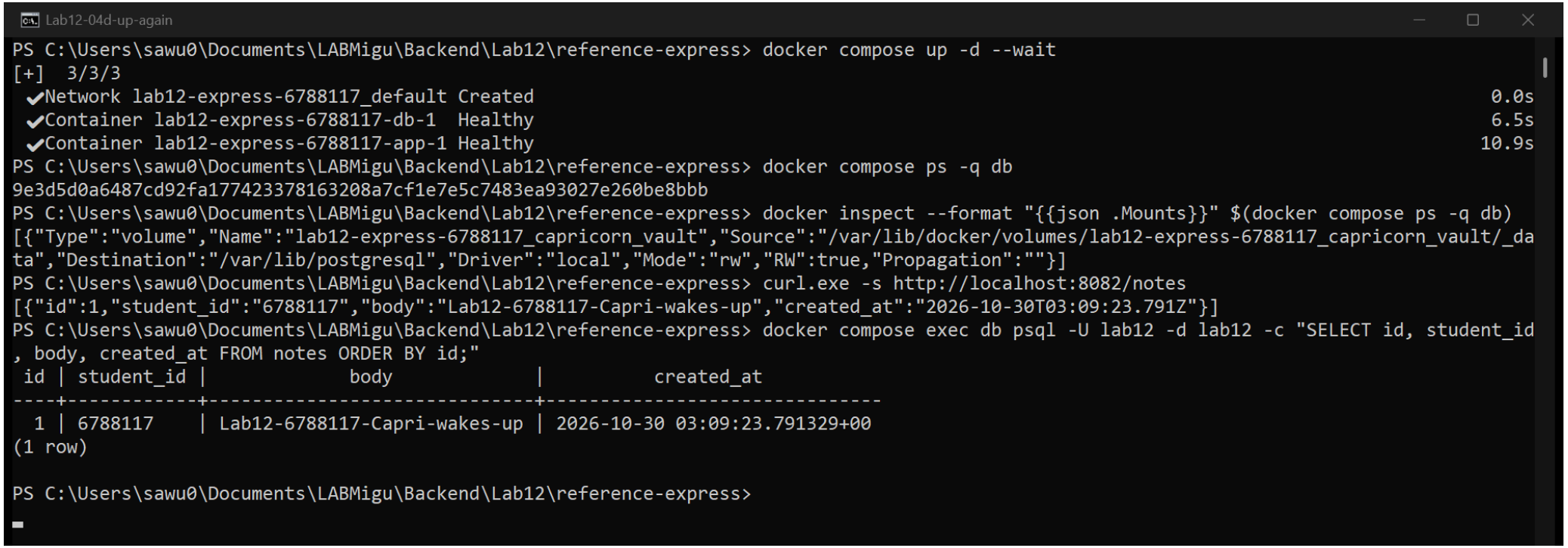
Before testing: DB container `9756324eaa3b...`. The mount is the named volume `lab12-express-6788117_capricorn_vault` at `/var/lib/postgresql`.



`docker compose restart` then up `-d --wait`. A restart stops and starts the same containers, so the ID is the same. API and SQL still show row 1.



`docker compose` down (no `-v`) removes both containers and the network. `ps -a` is empty, but the volume is still listed.



up -d --wait creates new containers. The DB container ID is now 9e3d5d0a6487..., but the mount is the same named volume. API and SQL show the same id, marker and created_at. I did not POST again.

Why the row survives. PostgreSQL writes its files to /var/lib/postgresql, and that path is the named volume, not the container's own writable layer. down deletes the container and its layer, but Compose never deletes named volumes on a normal down. The new container mounts the same volume and finds the old data files, so it skips init and serves the old rows.

down vs down -v. Plain down removes containers and the network and keeps named volumes. down -v also deletes the project's named volumes, so the data is gone. I did not run -v on my evidence data.

A volume is not a backup. It is a single copy on the same disk, inside the Docker Desktop VM. down -v, docker volume prune, resetting Docker Desktop, disk failure, or a bad DELETE all destroy it, and there is no older version to go back to. A backup is a separate copy kept somewhere else, for example pg_dump output saved outside Docker.

05 Own project

Project, framework and DB: Hotel Booking System API, NestJS 11 + Prisma 6, MySQL 8.4 (same engine and ORM as Lab 10/11), Redis for the room cache
Write/read routes: `POST /rooms` (ADMIN JWT) and `GET /rooms?search=Capri`, plus `POST /auth/register` and `POST /auth/login`
Ports: container 3000, host `127.0.0.1:8090`
Compose project: `lab12-project-6788117` (the reference is `lab12-express-6788117`)
Volume: `lab12-project-6788117_talia_safehouse` mounted at `/var/lib/mysql`

I copied my Lab 11 project into `Lab12/hotel-booking-api`. In Lab 11 the container used MySQL on my laptop through `host.docker.internal`. Now MySQL and Redis are Compose services and the app reaches them as `db` and `cache`. I added a `/health` route that runs `SELECT DATABASE(), NOW(3)` through Prisma, and a `migrate` stage in the Dockerfile so the schema is created with `prisma migrate deploy` before the API starts.

Compose file, Dockerfile and config

Lab12-05a-compose-file

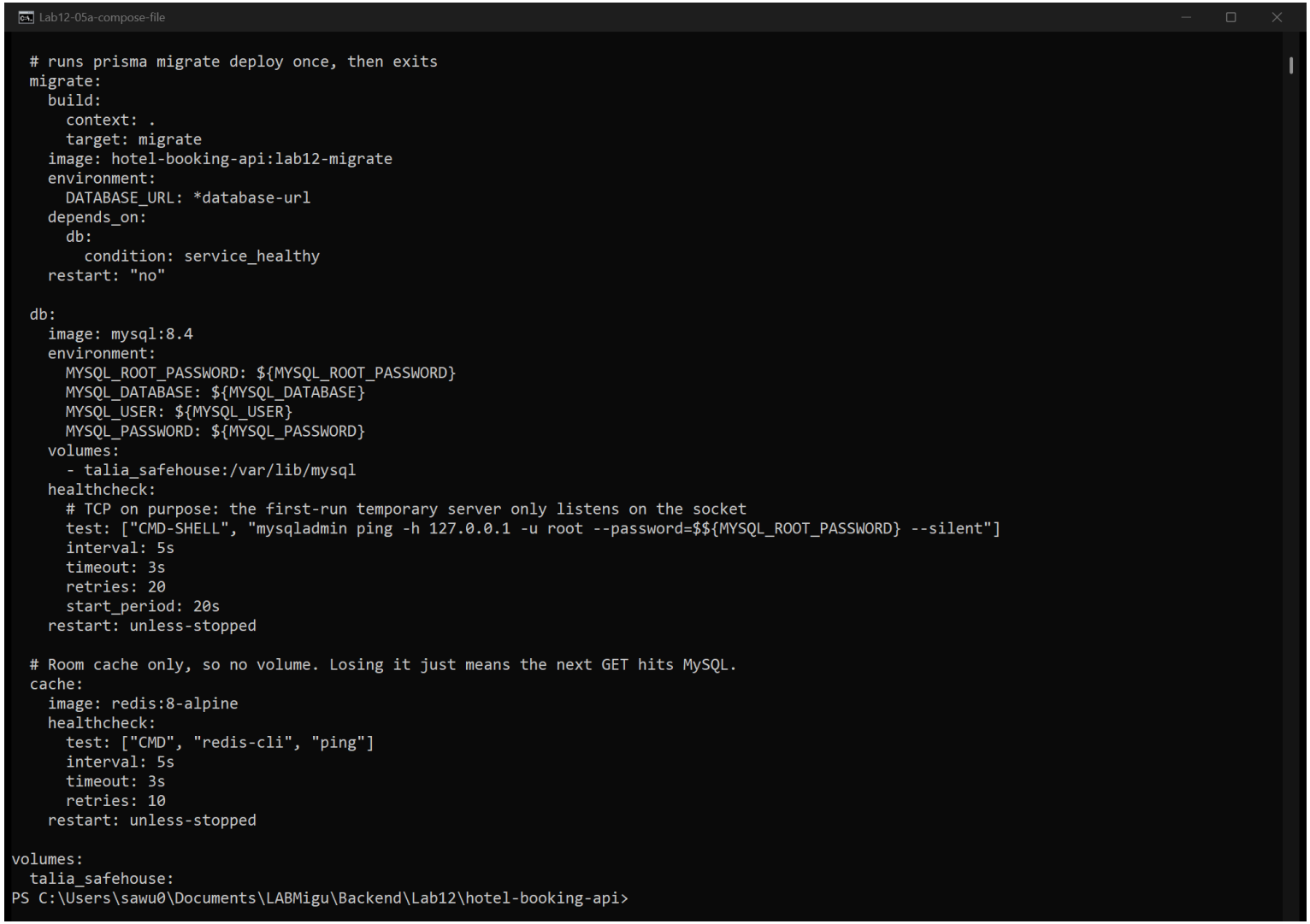
```
PS C:\Users\sawu0\Documents\LABMigu\Backend\Lab12\hotel-booking-api> Get-Content docker-compose.yml
name: lab12-project-${STUDENT_ID:?set STUDENT_ID in .env}

x-database-url: &database-url mysql://${MYSQL_USER}:${MYSQL_PASSWORD}@db:3306/${MYSQL_DATABASE}

services:
  app:
    build:
      context: .
      target: runtime
    image: hotel-booking-api:lab12
    ports:
      - "127.0.0.1:8090:3000"
    environment:
      PORT: 3000
      DATABASE_URL: *database-url
      REDIS_URL: redis://cache:6379
    depends_on:
      migrate:
        condition: service_completed_successfully
      cache:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "node", "-e", "fetch('http://127.0.0.1:3000/health').then(r => process.exit(r.ok ? 0 : 1)).catch(() => process.exit(1))"
]
      interval: 5s
      timeout: 3s
      retries: 10
      start_period: 10s
    restart: unless-stopped

  # runs prisma migrate deploy once, then exits
  migrate:
    build:
      context: .
      target: migrate
    image: hotel-booking-api:lab12-migrate
    environment:
      DATABASE_URL: *database-url
    depends_on:
      db:
        condition: service_healthy
    restart: "no"

  db:
    image: mysql:8.4
```



Four services. `migrate` waits for a healthy db, runs once and exits. app waits for `migrate` to complete successfully and for `cache` to be healthy. `DATABASE_URL` is built from `.env` values with host db. Redis has no volume because the cache can be rebuilt from MySQL.

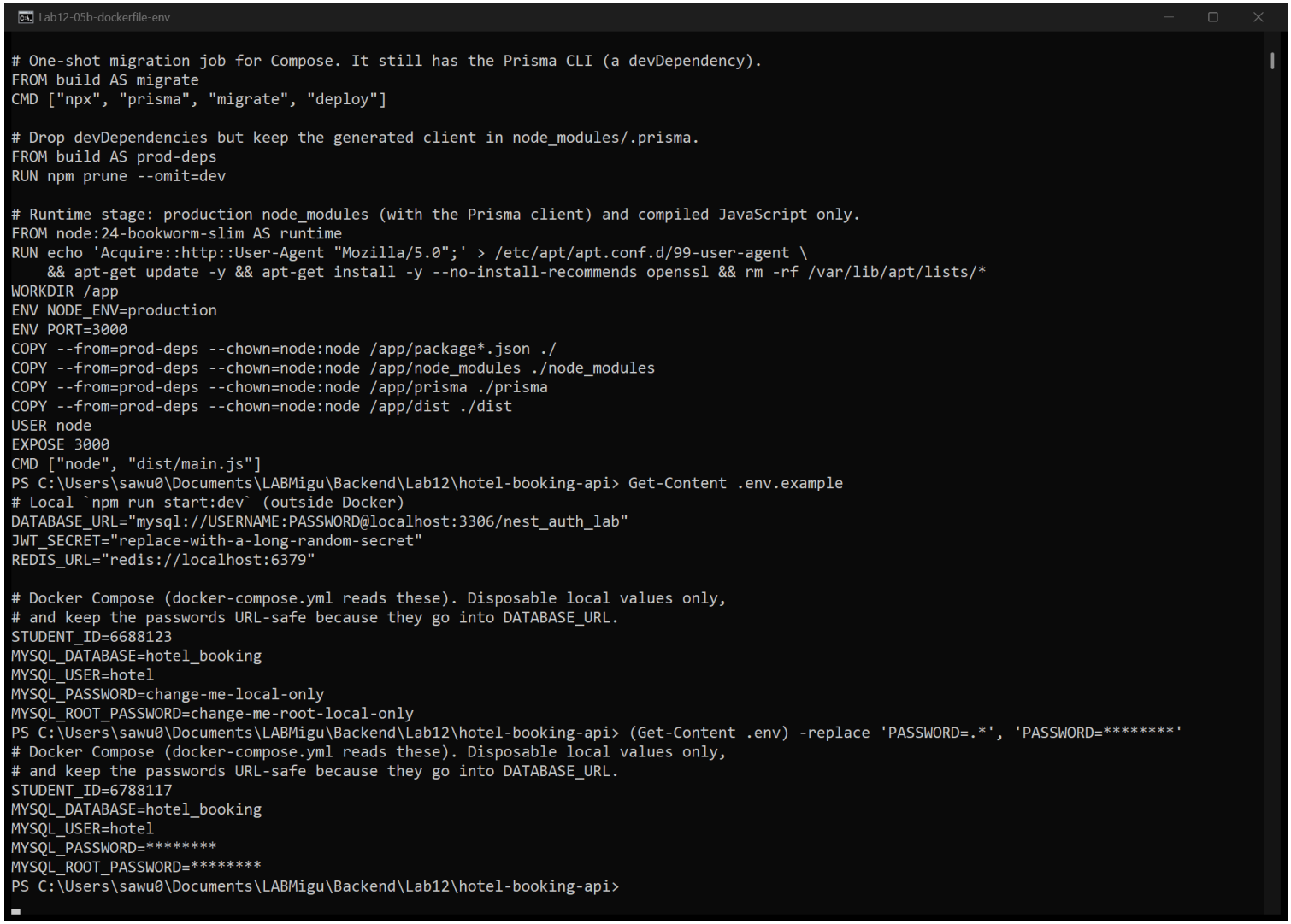
Lab12-05b-dockerfile-env

```
PS C:\Users\sawu0\Documents\LABMigu\Backend\Lab12\hotel-booking-api> Get-Content Dockerfile
# Build stage: install all dependencies, generate the Prisma client, compile TypeScript.
FROM node:24-bookworm-slim AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
# Prisma detects the OpenSSL version to pick its query engine; the slim image has none.
# My network drops requests with apt's own User-Agent ("Debian APT-HTTP"), so set a different one
# in apt.conf (an -o flag on "apt-get update" alone doesn't reach "apt-get install").
RUN echo 'Acquire::http::User-Agent "Mozilla/5.0";' > /etc/apt/apt.conf.d/99-user-agent \
    && apt-get update -y && apt-get install -y --no-install-recommends openssl && rm -rf /var/lib/apt/lists/*
# The postinstall "prisma generate" ran before the schema existed, so generate now.
COPY prisma ./prisma
RUN npx prisma generate
COPY tsconfig*.json nest-cli.json ./
COPY src ./src
RUN npm run build

# One-shot migration job for Compose. It still has the Prisma CLI (a devDependency).
FROM build AS migrate
CMD ["npx", "prisma", "migrate", "deploy"]

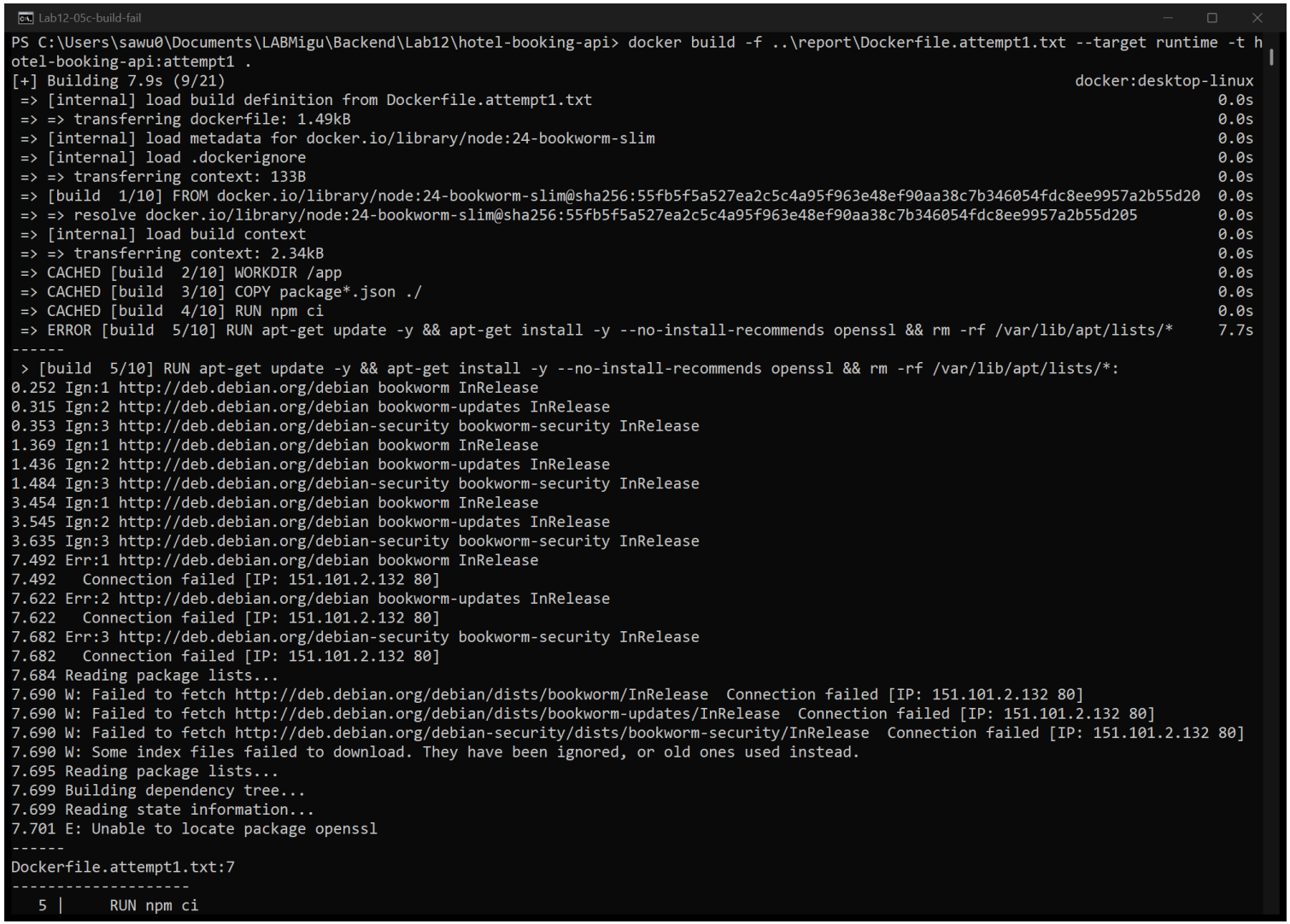
# Drop devDependencies but keep the generated client in node_modules/.prisma.
FROM build AS prod-deps
RUN npm prune --omit=dev

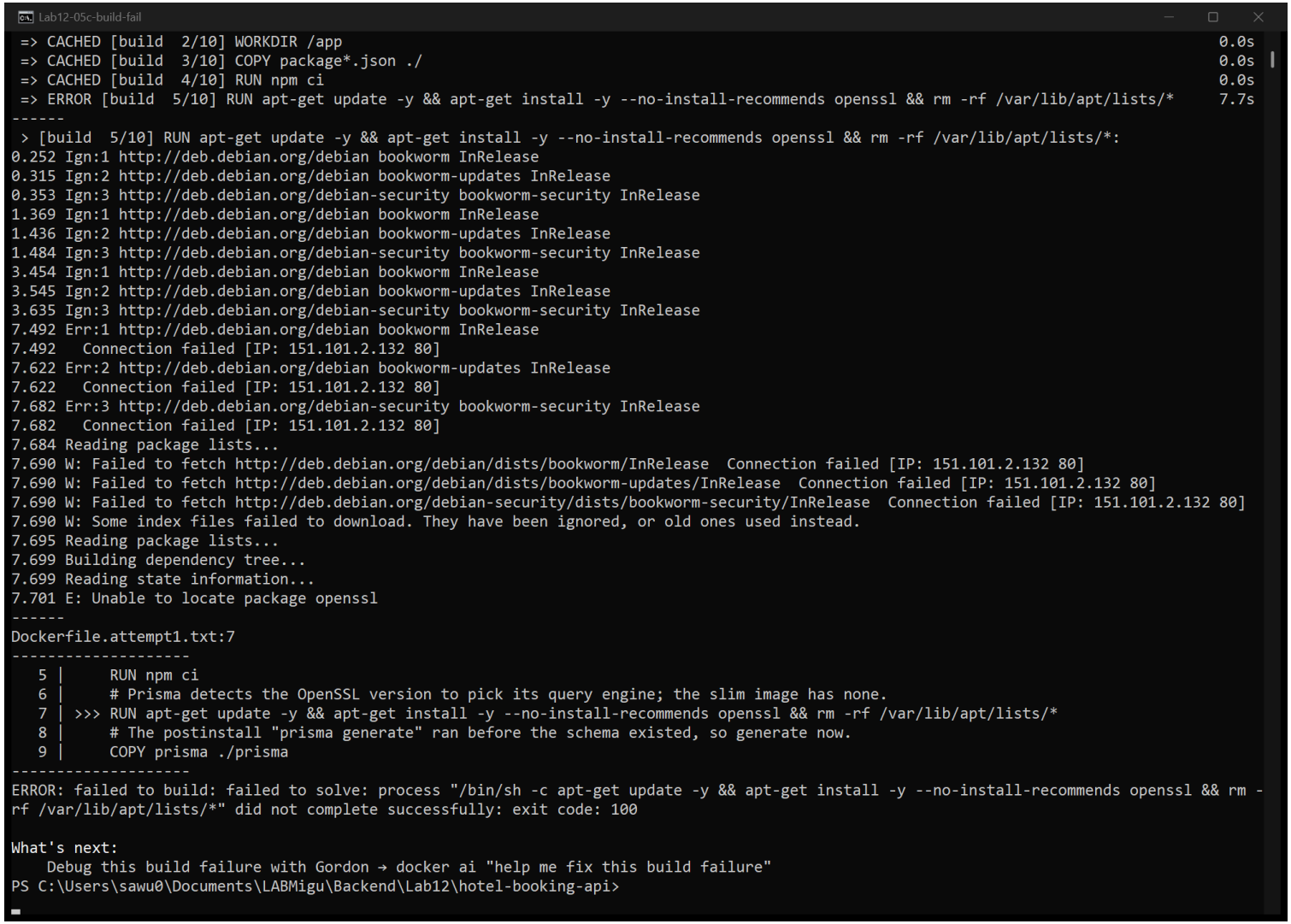
# Runtime stage: production node_modules (with the Prisma client) and compiled JavaScript only.
FROM node:24-bookworm-slim AS runtime
RUN echo 'Acquire::http::User-Agent "Mozilla/5.0";' > /etc/apt/apt.conf.d/99-user-agent \
    && apt-get update -y && apt-get install -y --no-install-recommends openssl && rm -rf /var/lib/apt/lists/*
WORKDIR /app
ENV NODE_ENV=production
ENV PORT=3000
COPY --from=prod-deps --chown=node:node /app/package*.json ./
COPY --from=prod-deps --chown=node:node /app/node_modules ./node_modules
COPY --from=prod-deps --chown=node:node /app/prisma ./prisma
COPY --from=prod-deps --chown=node:node /app/dist ./dist
USER node
EXPOSE 3000
CMD ["node", "dist/main.js"]
PS C:\Users\sawu0\Documents\LABMigu\Backend\Lab12\hotel-booking-api> Get-Content .env.example
# Local `npm run start:dev` (outside Docker)
DATABASE_URL="mysql://USERNAME:PASSWORD@localhost:3306/nest_auth_lab"
JWT_SECRET="replace-with-a-long-random-secret"
REDIS_URL="redis://localhost:6379"
```



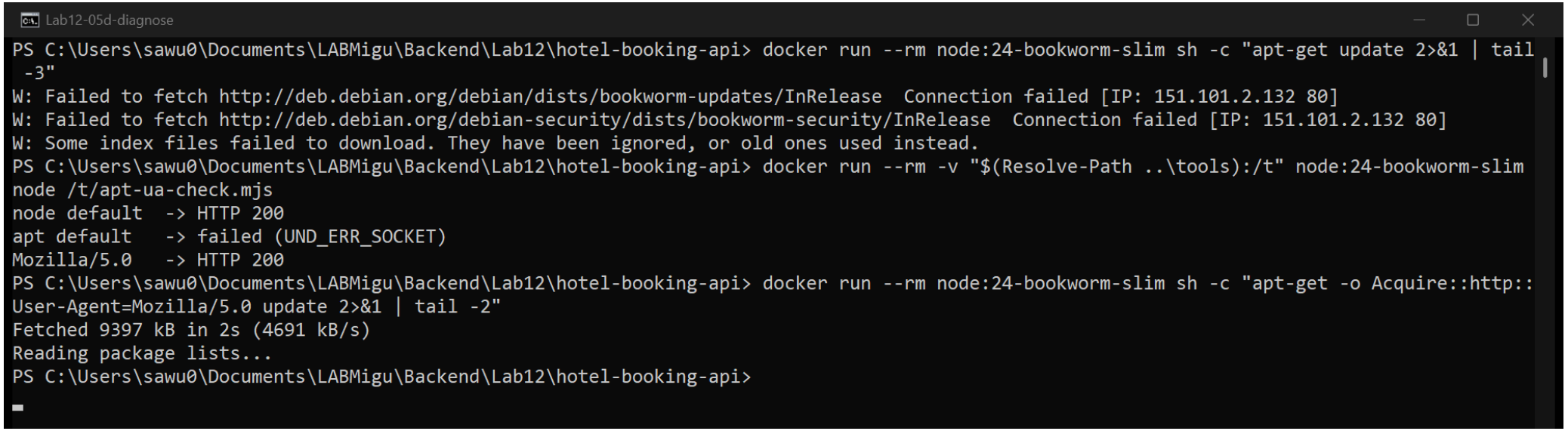
Dockerfile stages: `build`, `migrate` (still has the Prisma CLI), `prod-deps` (prunes dev packages) and `runtime`. `.env.example` has a local section and a Compose section, and the real `.env` has hidden passwords.

Problem: the image build failed





My first Compose build (the Lab 11 Dockerfile, kept as `report/Dockerfile.attempt1.txt`) failed at `apt-get update`: every request to `deb.debian.org` port 80 ended with `Connection failed`, so `E: Unable to locate package openssl`. The same Dockerfile built without problems in Lab 11.

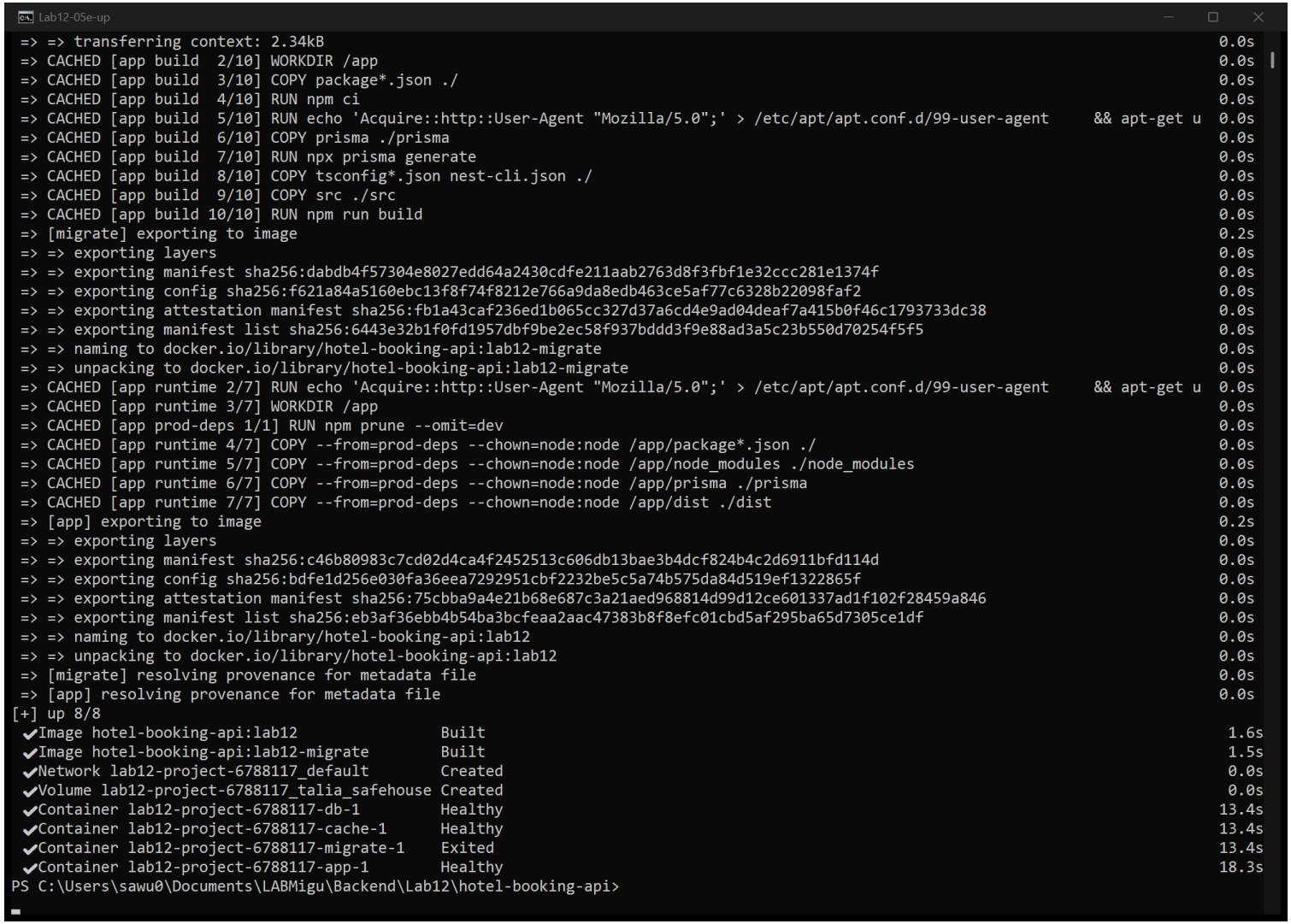


Diagnosis. Plain `apt-get update` fails in a normal container too, so it is not Compose. I wrote `tools/apt-ua-check.mjs` to fetch the same Debian URL with different User-Agents: Node's default and `Mozilla/5.0` get HTTP 200, but apt's own `Debian APT-HTTP/1.3` is cut off. So my network drops requests with apt's User-Agent. With `-o Acquire::http::User-Agent=Mozilla/5.0`, `apt-get update` works.

Fix. My first try put `-o Acquire::http::User-Agent=...` only on `apt-get update`. The index downloaded, but `apt-get install` still used the default agent and failed on the `.deb` files. So I put the setting in `/etc/apt/apt.conf.d/99-user-agent`, which both commands read, in the build and runtime stages.

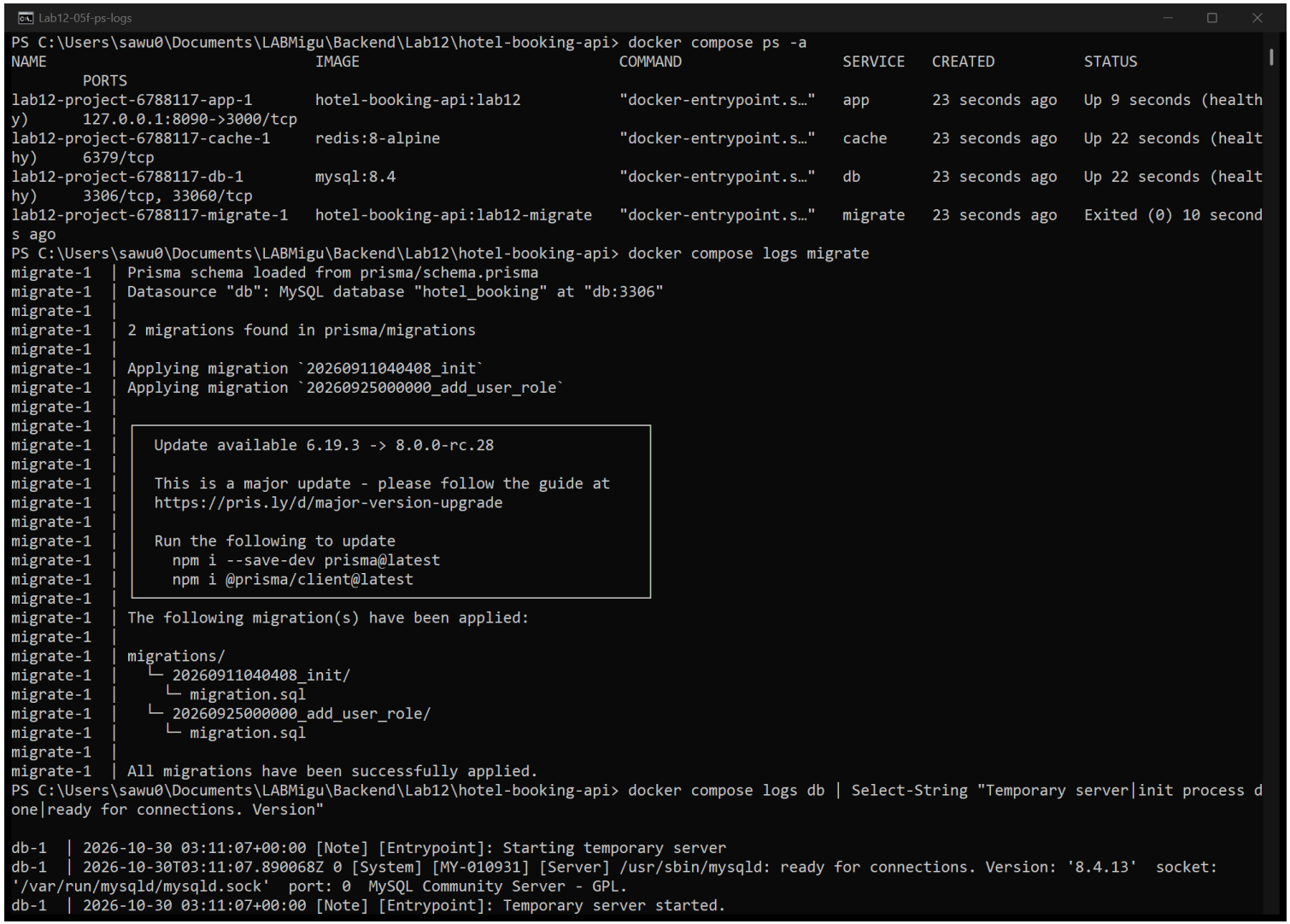
Lab12-05e-up

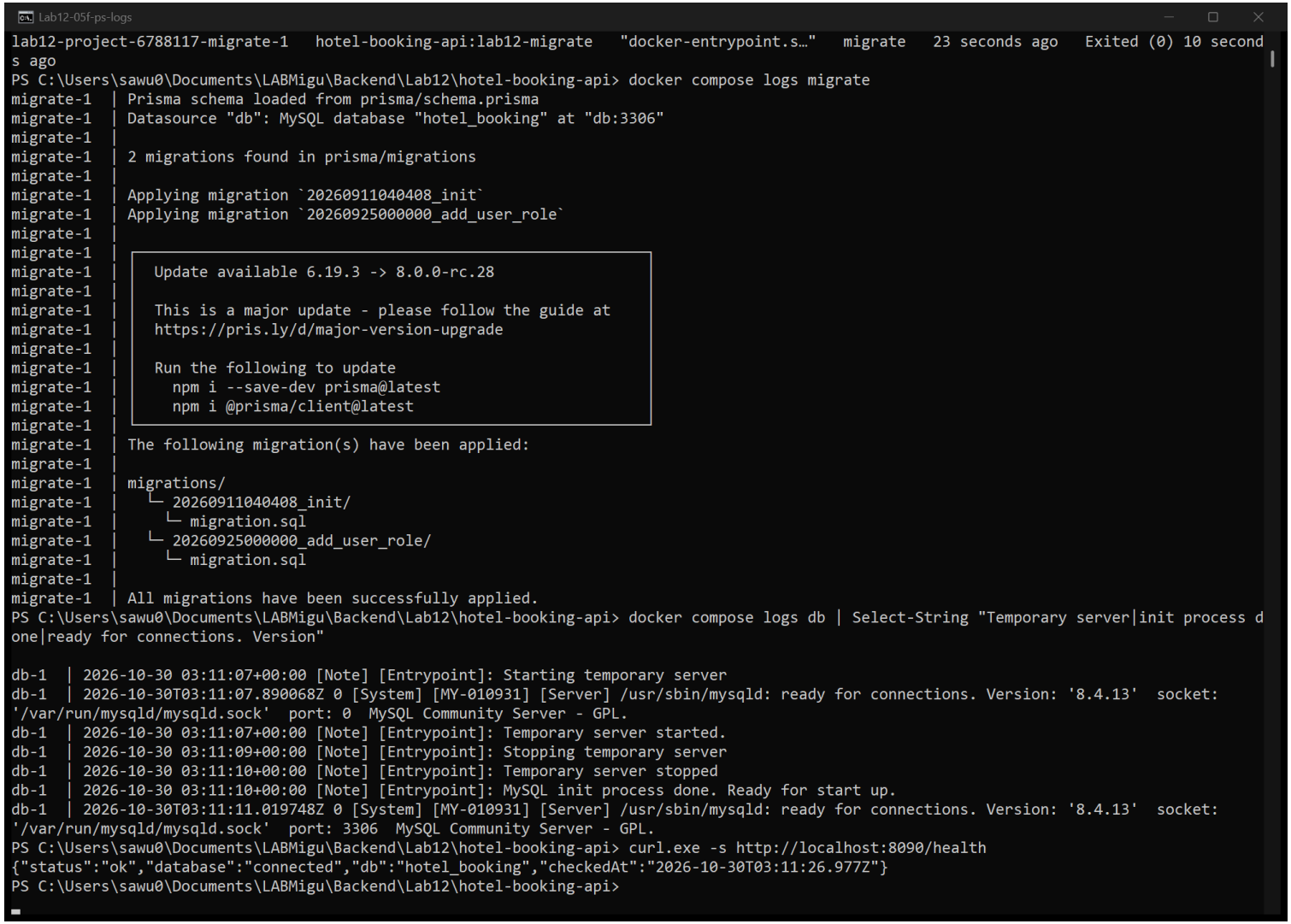
```
PS C:\Users\sawu0\Documents\LABMigu\Backend\Lab12\hotel-booking-api> docker compose config --quiet; if ($?) { "compose config OK" }
compose config OK
PS C:\Users\sawu0\Documents\LABMigu\Backend\Lab12\hotel-booking-api> docker compose up -d --build --wait
[+] Building 1.3s (26/26) FINISHED
 => [internal] load local bake definitions                                                                                0.0s
 => => reading from stdin 1.21kB                                                                                          0.0s
 => [migrate internal] load build definition from Dockerfile                                                              0.0s
 => => transferring dockerfile: 1.89kB                                                                                    0.0s
 => [migrate internal] load metadata for docker.io/library/node:24-bookworm-slim                                          0.0s
 => [migrate internal] load .dockerignore                                                                                 0.0s
 => => transferring context: 133B                                                                                         0.0s
 => [app build  1/10] FROM docker.io/library/node:24-bookworm-slim@sha256:55fb5f5a527ea2c5c4a95f963e48ef90aa38c7b346054fdc8ee9957a2b5  0.0s
 => => resolve docker.io/library/node:24-bookworm-slim@sha256:55fb5f5a527ea2c5c4a95f963e48ef90aa38c7b346054fdc8ee9957a2b55d205  0.0s
 => [migrate internal] load build context                                                                                 0.0s
 => => transferring context: 2.34kB                                                                                       0.0s
 => CACHED [app build  2/10] WORKDIR /app                                                                                 0.0s
 => CACHED [app build  3/10] COPY package*.json ./                                                                        0.0s
 => CACHED [app build  4/10] RUN npm ci                                                                                   0.0s
 => CACHED [app build  5/10] RUN echo 'Acquire::http::User-Agent "Mozilla/5.0";' > /etc/apt/apt.conf.d/99-user-agent     && apt-get u  0.0s
 => CACHED [app build  6/10] COPY prisma ./prisma                                                                         0.0s
 => CACHED [app build  7/10] RUN npx prisma generate                                                                      0.0s
 => CACHED [app build  8/10] COPY tsconfig*.json nest-cli.json ./                                                         0.0s
 => CACHED [app build  9/10] COPY src ./src                                                                               0.0s
 => CACHED [app build 10/10] RUN npm run build                                                                            0.0s
 => [migrate] exporting to image                                                                                          0.2s
 => => exporting layers                                                                                                   0.0s
 => => exporting manifest sha256:dabdb4f57304e8027edd64a2430cdfe211aab2763d8f3fbf1e32ccc281e1374f                         0.0s
 => => exporting config sha256:f621a84a5160ebc13f8f74f8212e766a9da8edb463ce5af77c6328b22098faf2                           0.0s
 => => exporting attestation manifest sha256:fb1a43caf236ed1b065cc327d37a6cd4e9ad04deaf7a415b0f46c1793733dc38             0.0s
 => => exporting manifest list sha256:6443e32b1f0fd1957dbf9be2ec58f937bddd3f9e88ad3a5c23b550d70254f5f5                    0.0s
 => => naming to docker.io/library/hotel-booking-api:lab12-migrate                                                        0.0s
 => => unpacking to docker.io/library/hotel-booking-api:lab12-migrate                                                     0.0s
 => CACHED [app runtime 2/7] RUN echo 'Acquire::http::User-Agent "Mozilla/5.0";' > /etc/apt/apt.conf.d/99-user-agent     && apt-get u  0.0s
 => CACHED [app runtime 3/7] WORKDIR /app                                                                                 0.0s
 => CACHED [app prod-deps 1/1] RUN npm prune --omit=dev                                                                   0.0s
 => CACHED [app runtime 4/7] COPY --from=prod-deps --chown=node:node /app/package*.json ./                                0.0s
 => CACHED [app runtime 5/7] COPY --from=prod-deps --chown=node:node /app/node_modules ./node_modules                     0.0s
 => CACHED [app runtime 6/7] COPY --from=prod-deps --chown=node:node /app/prisma ./prisma                                 0.0s
 => CACHED [app runtime 7/7] COPY --from=prod-deps --chown=node:node /app/dist ./dist                                     0.0s
 => [app] exporting to image                                                                                              0.2s
 => => exporting layers                                                                                                   0.0s
 => => exporting manifest sha256:c46b80983c7cd02d4ca4f2452513c606db13bae3b4dcf824b4c2d6911bfd114d                         0.0s
 => => exporting config sha256:bdfe1d256e030fa36eea7292951cbf2232be5c5a74b575da84d519ef1322865f                           0.0s
 => => exporting attestation manifest sha256:75cbba9a4e21b68e687c3a21aed968814d99d12ce601337ad1f102f28459a846             0.0s
 => => exporting manifest list sha256:eb3af36ebb4b54ba3bcfeaa2aac47383b8f8efc01cbd5af295ba65d7305ce1df                    0.0s
 => => naming to docker.io/library/hotel-booking-api:lab12                                                                0.0s
```



With the fix, `docker compose up -d --build --wait` builds both images (`lab12` and `lab12-migrate`) and creates the volume `lab12-project-6788117_talia_safehouse`. db and cache are Healthy, `migrate` Exited and app Healthy.

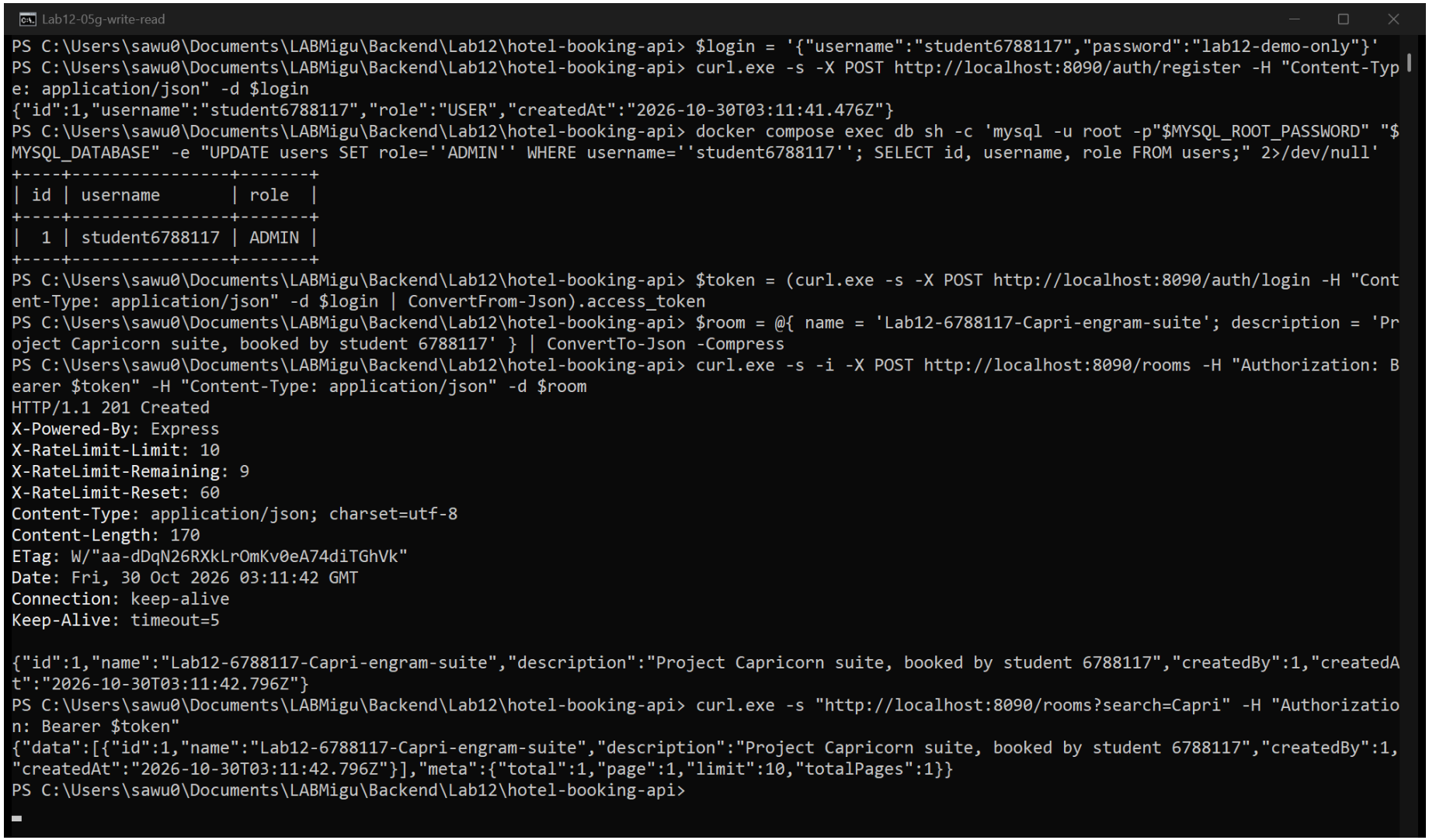
Preventive check: when is MySQL really ready?



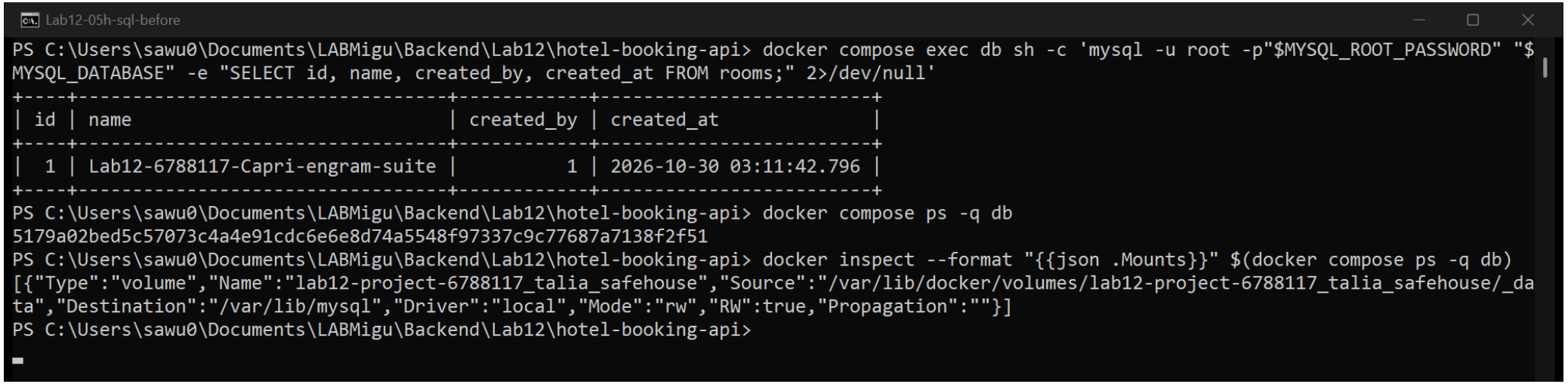


`migrate` exited with code 0 after applying both migrations to `hotel_booking` at `db:3306`. The DB log shows why my healthcheck uses `mysqladmin ping -h 127.0.0.1`: on first start MySQL runs a temporary server with `port: 0` (socket only), stops it, and then starts the real server on `port: 3306`. A socket ping would pass during the temporary server and `migrate` would fail to connect. `/health` reports the DB as connected.

Write and read through the real API

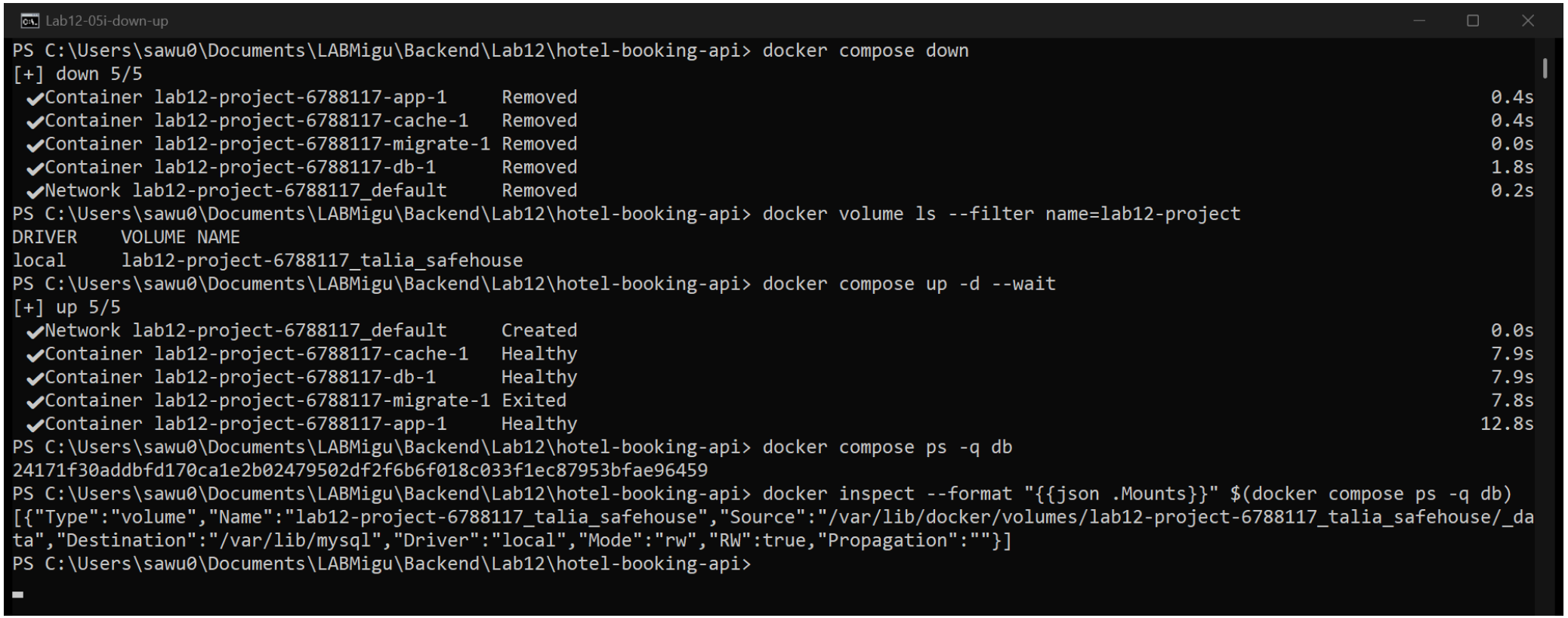


Register `student6788117` (USER), promote it to ADMIN with SQL inside the db container (register never accepts a role), log in, then POST `/rooms` returns **201** with room id 1, `Lab12-6788117-Capri-engram-suite`, createdAt `2026-10-30T03:11:42.796Z`. GET `/rooms?search=Capri` returns it. The password is a throwaway value that only exists in this local Compose database.

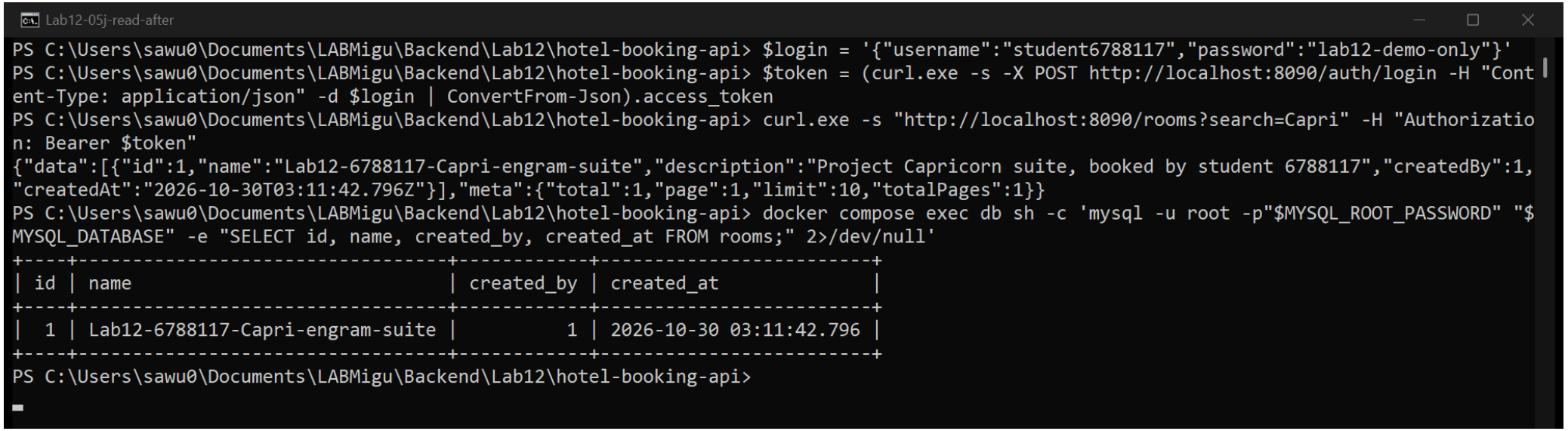


SQL check: the same row in MySQL (created_by 1). DB container `5179a02bed5c...` with the volume `lab12-project-6788117_talia_safehouse` at `/var/lib/mysql`.

Persistence after down/up

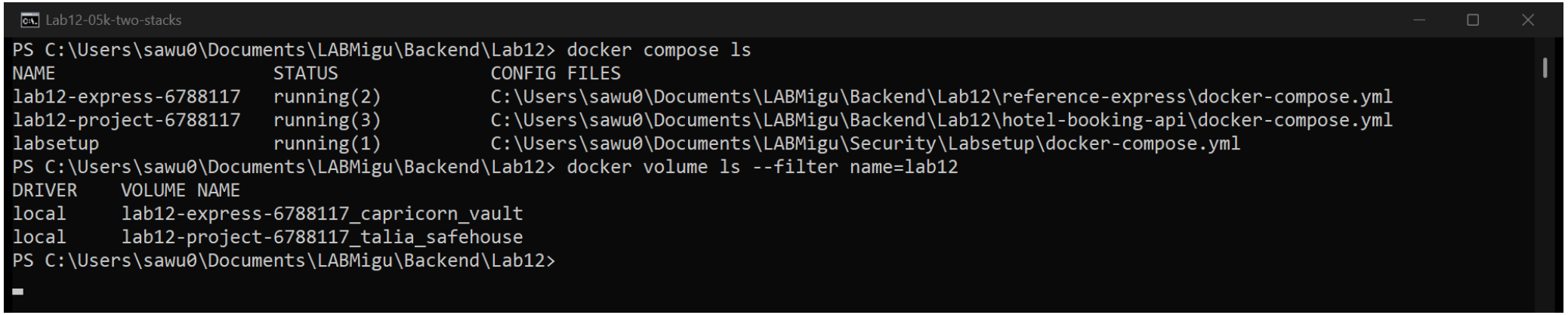


`docker compose down` removes all four containers. The volume is still there. After `up -d --wait`, the DB container is new (`24171f30addb...`) but the mount is the same named volume. `migrate` ran again and exited before the app started.

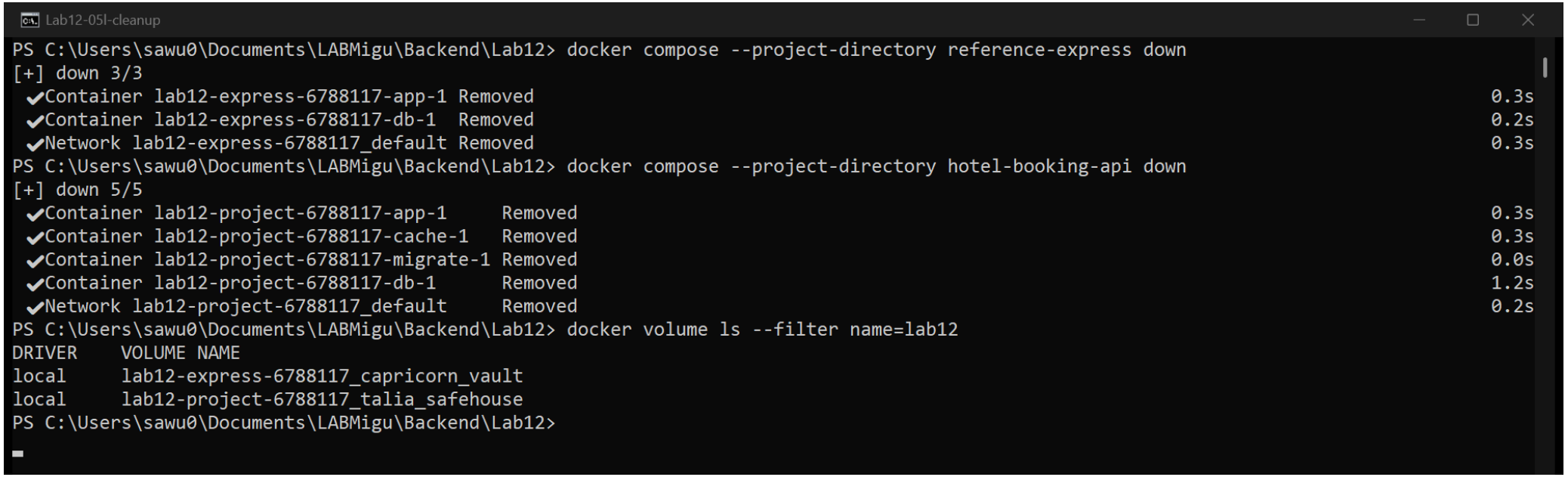


After recreation I only logged in and read. API and SQL show the same room id 1, name and created_at (`03:11:42.796`). Redis was recreated empty, so this GET came from MySQL, not from an old cache entry.

Test	DB container ID	Named volume	Record ID	Marker + created_at
Before	`5179a02bed5c`	`lab12-project-6788117_talia_safehouse`	1	`Lab12-6788117-Capri-engram-suite, 2026-10-30T03:11:42.796Z`
After down/up	`24171f30addb (new)`	`lab12-project-6788117_talia_safehouse`	1	`Lab12-6788117-Capri-engram-suite, 2026-10-30T03:11:42.796Z`



Both stacks have their own project name and volume, so my project never reuses the reference DB. (`labsetup` is from another course.)



Cleanup with plain `docker compose down` for both projects. Both volumes are kept.

Startup instructions

`Copy-Item .env.example .env` and set `STUDENT_ID` and throwaway, URL-safe MySQL passwords
`docker compose config --quiet`
`docker compose up -d --build --wait`
`docker compose ps -a` (migrate `Exited (0)`, the others healthy) and `curl.exe -s http://localhost:8090/health`
Stop with `docker compose down` (never `-v` on data you need)

Lab 13 mapping

Locally I have one API container and one MySQL container on one machine. In Lab 13 the API image (the `runtime` target) becomes the application service, and MySQL should become a database service with its own storage and backups instead of a container volume on my laptop. Values from `.env` become settings or secrets on the platform, and `DATABASE_URL` will point to the cloud database host instead of `db`. The `migrate` step has to run against that database before the new API version starts. Redis can stay a cache service. My local rows will not appear online by themselves. The volume stays on my laptop, so I would need to export the data (for example with `mysqldump`) and import it, or start online with an empty database.